

昨天你看着 Engine 跑——如果你不在呢？

L3 · 人驱动（昨天）

- 你下发任务 → agent 按阶段走
- 你看着 → 纠正偏差 → 走完
- 问题：你是瓶颈，你不在就跑不了

L4 · 自驱动（目标）

- 你设意图 → agent 自主跑 → 你偶尔检查
- Engine 需要：知道何时停 / 何时求助 / 何时放弃
- Loop Engineering = 给 Engine 装「自动驾驶」

Loop = 停止条件 + 熔断机制 + 成本治理 + 升级策略

什么算「完成了」——不是 agent 说了算

如果让 agent 自己判断「完成了吗」——它永远觉得「差不多了」。停止条件必须是外部可验证的。

类型	例子	可判定吗
目标达成	所有 AC 标记 pass (G4 通过)	✓
时间限制	运行超过 60 min	✓
成本限制	token 消耗超过 \$5	✓
循环限制	同一 gate 失败超过 5 次	✓
质量阈值	eval score $\geq 80\%$	✓
「感觉差不多了」	自我判断	✗ 不可判定

engine/loop-config.md

```
## 停止条件 (满足任一即停)
- goal: 所有 AC pass (G4)
- timeout: 60 min
- cost: $5 / 50K tokens
- max_retries: gate 失败 5 次
- eval_threshold: score  $\geq 80\%$ 
```

停止条件外部化 —— 不是 agent 说完了就完了，是外部条件满足了才停。
哪个条件先触发就停在哪，剩下的由你判断「接下来怎么办」。

什么时候必须紧急停下 —— 熔断器

需要熔断的情况



无限循环

连续 N 次走同一路径无进展 (3 次相同 pattern)



成本爆炸

token 速率异常 (>10K tokens/min 持续 3 min)



破坏行为

试图删除大量文件 / 修改安全配置 / rm -rf



外部异常

API 连续失败 / 网络断开 (连续 5 次)

熔断后的处理

- 立即停止所有活动
- 保存当前状态到 STATE.md
- 输出诊断信息 (停在哪、为什么)
- 等待人介入 (不自动恢复)

正常停 · 停止条件

紧急停 · 熔断

正常结束

异常中断

可自动开始下一任务

必须人来看

Loop 的隐性成本 —— Token 不是免费的

Loop 的成本现实

一次 feature 跑完 $\approx$ 50–200K tokens
(含设计 + 编码 + 测试 + 验证)
有重试 (gate 失败) $\rightarrow$ 可能 300K+
10 个任务的 batch $\approx$ 500K–2M tokens
一晚上无约束 loop $\rightarrow$ 可能 \$100–300

效率指标: tokens / AC

✅ 好的效率: 5–10K tokens 完成一个 AC

❌ 差的效率: 30K+ tokens 还在循环

效率太差 $\rightarrow$ agent 可能在做无效工作

策略	做法	效果
预算帽	单任务 \$5 / 批量 \$20	硬限制
渐进放权	小任务先跑, 成功后放大	降低风险
效率监控	tokens/AC (每个验收标准消耗)	发现浪费

成本意识是 L4 的一部分 —— 不只是「能不能自动跑」, 还有「跑得值不值」。

loop-config.md —— 把所有约束写在一个地方

```
# engine/loop-config.md
```

停止条件 (满足任一即停)

- goal_achieved: G4 通过 (所有 AC pass)
- timeout: 60 min per task
- cost_cap: \$5 per task / \$20 per batch
- max_gate_retries: 5 - eval_threshold: score $\geq$ 80%

熔断条件 (立即中断 + 等人)

- infinite_loop: 连续 3 次完全相同的 gate 失败 pattern
- cost_spike: >10K tokens/min 持续 3 min
- destructive_action: 任何 rm -rf / permission / data delete
- api_failure: 连续 5 次 API 异常

成本治理

- budget_per_task: \$5 - budget_per_session: \$20
- efficiency_alert: >15K tokens/AC
- report_after: 每个任务完成后输出成本报告

升级策略 (什么时候找人)

- gate_fail_3x: 同一 gate 连续 3 次失败 → notify
- new_error_type: corrections 出现从未见过的类别 → notify
- architecture_decision: PLAN 阶段 ADR → 等待审批 (L3 gate)

停止条件

正常退出

熔断

异常退出

成本

预算约束

升级

什么时候找你

Loop ≠ 「什么都自己决定」：不可逆决策仍需你批准。

不要一步到 L4 —— 渐进放权



信任不是配置出来的——是赢得的

corrections 频率下降
= 信任上升 = 可降级 gate

corrections 突然上升
= 信任下降 = 升级回来

这个过程可能1个月也可能3个月
—— 取决于项目复杂度和agent 的学习速度。

动手：写 loop-config + 升级策略

Lab 任务 (20 min)

1 创建 engine/loop-config.md

停止条件 (至少 3 个) · 熔断条件 (至少 2 个) · 成本预算 · 升级策略

2 更新 engine/gates.md

为每个 gate 标注「当前级别」和「降级条件」

3 思考两个问题

你的第一个 loop 实验会是什么？ (最安全的任务类型)

什么任务你永远不会放进 loop？ (需要人判断的)

✓ 第一个 loop 选什么

bugfix 类 —— 方向明确、风险低、可验证。

✗ 永远不放进 loop

涉及不可逆决策 —— 新项目架构选型、数据迁移。

Loop = Engine + 停止条件 + 熔断 + 成本治理



停止条件

目标 / 时间 / 成本 / 循环 / 质量阈值



熔断机制

无限循环 / 成本爆炸 / 破坏 / API 异常



成本治理

预算帽 + 效率监控 + 渐进放权



升级策略

什么时候找人 · L3 → L4 是赢得的

Loop 不是「放飞」—— 是「有约束的自主」